

Tensor Casting: Co-Designing Algorithm-Architecture for Personalized Recommendation Training

Youngeun Kwon
School of Electrical Engineering
KAIST
yekwon@kaist.ac.kr

Yunjae Lee
School of Electrical Engineering
KAIST
yunjae408@kaist.ac.kr

Minsoo Rhu
School of Electrical Engineering
KAIST
mrhu@kaist.ac.kr

Abstract—Personalized recommendations are one of the most widely deployed machine learning (ML) workload serviced from cloud datacenters. As such, architectural solutions for high-performance recommendation inference have recently been the target of several prior literatures. Unfortunately, little have been explored and understood regarding the training side of this emerging ML workload. In this paper, we first perform a detailed workload characterization study on training recommendations, root-causing sparse embedding layer training as one of the most significant performance bottlenecks. We then propose our algorithm-architecture co-design called Tensor Casting, which enables the development of a generic accelerator architecture for tensor gather-scatter that encompasses all the key primitives of training embedding layers. When prototyped on a real CPU-GPU system, Tensor Casting provides $1.9 - 21\times$ improvements in training throughput compared to state-of-the-art approaches.

Index Terms—computer architecture, memory-centric, machine learning, deep learning, recommendation system

I. INTRODUCTION

The enormous compute power machine learning (ML) accelerators (also known as *tensor processing units*, TPUs) deliver allows practitioners to develop sophisticated and powerful deep neural network (DNN) algorithms. Such trend has spurred numerous academic and industrial proposals on designing energy-efficient TPUs for both training and inference. Interestingly, while it is challenging to define a *generic* TPU architecture that encompasses numerous design points for ML inference (i.e., they are typically optimized for a *diverse* set of compute primitives that represent a specific application domain, for instance, convolutions for computer vision [6], [7], [22], [49], [54]), TPUs for training have more or less settled on a design optimized for a *single* key primitive: general-purpose matrix multiplication (GEMM). Such trend is notably represented by major vendors in the ML training market optimizing their TPU for high-performance GEMM operations, such as NVIDIA’s Tensor Core [43], Google’s TPU [30], Habana’s Gaudi [21], Graphcore’s IPU [16], and Cerebra’s CS-1 [55]. The reason why TPUs for training are optimized for this single compute primitive is because of the *versatility* of GEMM operators. Concretely, the backpropagation based training algorithm [38] requires the derivation of input and weight gradients, both of which are able to be *casted* as a GEMM operation for popular DNN layers, such as convolutional, fully-connected, and recurrent layers. As these GEMM-compatible operations cover a significant portion of the training time for

conventional DNNs, the design cost of a TPU can be amortized by optimizing its microarchitecture for this all-round compute primitive. As such, we have witnessed great success of these GEMM-optimized TPUs for the purpose of training DNNs for computer vision, natural language processing, etc.

Despite such success however, emerging ML applications are now evolving into having complex DNN topologies with algorithmic components that cannot be singlehandedly accelerated with these GEMM-optimized TPUs. For instance, ML algorithms for natural language processing [12], memory-augmented neural networks [53], or recommender systems [11], [23], [41] employ “sparse” *embedding layers* whose compute and memory access characteristics significantly differ compared to conventional “dense” DNN layers. In particular, personalized recommendation models for consumer facing products (e.g., e-commerce, Ads) stand out with their high memory capacity and bandwidth demands [19], [32], [35], rendering conventional dense-optimized TPUs suboptimal in handling the training process of recommendations. Because of the *embedding tables* they utilize, the memory capacity demands of state-of-the-art recommendation models are in the order of several tens to thousands of GBs, far exceeding the memory capabilities of TPU devices. Consequently, existing solutions for recommendations almost exclusively utilize the host CPU memory [14], [57] as a placeholder to store these embedding tables to overcome the limited memory size of TPUs. Facebook’s Zion large-memory unified training platform [14] for instance employs a large pooled CPU memory for storing memory-capacity limited embedding tables, with Baidu’s AIBox [57] targeting recommendation models requiring several TBs of storage. Now, an interesting property of training recommendation models is that these *memory-capacity limited embedding tables are the model parameters that are subject for training*. Therefore, training recommendation models takes a *hybrid* approach where the sparse embedding layers are trained by the CPU while the dense DNN layers are trained using the TPU. As we detail in Section III-A, such *CPU-centric* training of embeddings (i.e., the key compute primitives of both forward and backward propagation of embedding layers are executed by the CPU) causes serious system-level bottlenecks because of the low compute and memory throughput of CPUs, with the TPU contributing to less than 6% of the training time for several

embedding intensive models. This raises questions on the cost-effectiveness of dense TPUs for training recommendations, as the majority of training time can be spent “outside” the TPU accelerator, for instance, the CPUs.

Unfortunately, little has been studied and understood in the computer systems community regarding the algorithmic properties of training DNN-based recommendation models let alone the accelerator system architecture that enables it. As such, an important motivation and **key contribution** of our study is a detailed analysis and characterization of the training process of DNN-based recommendations. Several recent work [32], [35] identified and addressed the performance bottlenecks caused by the *tensor gather-and-reduce* operation, a key primitive of the “inference” pass of recommendations. A unique contribution of our characterization is the revelation of a new system-level bottleneck in “training” recommendations, the *gradient expand-and-coalesce* operator, which is a highly throughput-hungry compute primitive undertaken by the CPU during the backpropagation stage of embedding layers.

To address the system-level bottlenecks of baseline CPU-centric systems, we propose an algorithm-architecture co-design for high-throughput recommendation training. The proposed co-design is based on our novel *Tensor Casting* algorithm that transforms the backpropagation’s gradient expand-and-coalesce primitive into a tensor gather-reduce operation¹ which is, as noted above, the key compute primitive of the forward propagation of embedding layers (we detail why Tensor Casting transforms it into a gather-reduce later in this section). The **key innovation** of our Tensor Casting is twofold. First, Tensor Casting algorithmically guarantees that the memory intensity of the gradient expand-and-coalesce operation is reduced by $2\times$ when it is casted to a tensor gather-reduce operator, significantly reducing the latency in conducting backpropagation of embedding layers. Additionally, Tensor Casting utilizes the current software stack as-is so another key advantage of our proposal is that it can be implemented purely in software. We implement Tensor Casting using PyTorch [50] and CUDA, quantitatively demonstrating that our proposal improves the end-to-end training performance of current CPU-centric systems by $1.2\text{--}2.8\times$.

While the performance improvement offered by our Tensor Casting algorithm is impressive, we observe that the baseline CPU-centric system does not fully reap out the potential of our proposal. To maximally utilize the opportunities inherent with our algorithm-architecture co-design, the second key innovation of our work is the development of a *memory-centric* training system for recommendations. Recall that the tensor gather-reduce and the gradient expand-and-coalesce operators are the primary computational building block of the forward and backward propagation of embedding layers, respectively. When Tensor Casting is used to transform the gradient expand-and-coalesce into a tensor gather-reduce operator, “both” forward and backward propagation are now

¹As gradients are structured as tensors, we interchangeably refer to *casted* operation as both tensor gather-reduce and gradient gather-reduce.

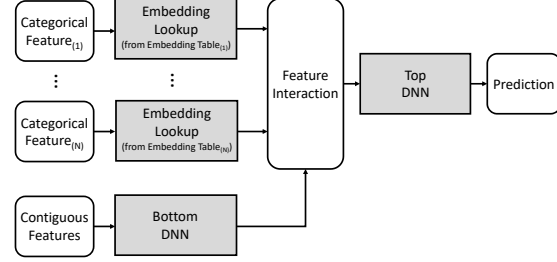


Fig. 1: Topological structure of a DNN-based recommendation model.

able to be executed using a *common* compute primitive: tensor gather-reduce. This opens up a unique opportunity to design a *generic* accelerator microarchitecture that covers both the forward and backpropagation of embedding layers.

As such, our development of Tensor Casting to permute gradient expand-and-coalesce into tensor gather-reduce is intentional: similar to how the GEMM-optimized TPU architecture has been instrumental in the success of training *dense* and *compute-intensive* DNNs, we argue that emerging ML workloads using embeddings deserve a targeted acceleration treatment tailored to the *sparse* and *memory-intensive* dataflow of training embedding layers. Building on top of a recently proposed near-memory processing (NMP) solution for recommendation inference [32], [35], the novelty of our memory-centric approach is the utilization of the *expressive power of the tensor gather-reduce* operator to develop a generic, *sparse*-optimized NMP accelerator for training embedding layers. We demonstrate that our NMP accelerator for tensor gather-reduce can accelerate the most time-consuming primitives of recommendation training. The effectiveness of our memory-centric system is demonstrated as a proof-of-concept prototype on a high-end GPU system, achieving an additional $1.5\text{--}16\times$ training time reduction than our software-only CPU-centric system optimized with Tensor Casting ($1.9\text{--}21\times$ speedup when compared against the baseline CPU-centric systems).

II. BACKGROUND AND RELATED WORK

A. Embedding Layers in Recommendations

The ability to “learn” a semantically meaningful representation of a target feature is one of the biggest strengths of ML. Rather than using one-hot encoded vectors for representing categorical features, recent work [12], [41], [53] has shown that “*embeddings*” can be used for learning the semantic representation of categorical data. For example, different movies serviced by online video streaming service (e.g., YouTube, Netflix) are projected from a discrete movie ID (i.e., each movie is assigned with a unique ID) to a corresponding continuous vector representation using the *embedding layers*. An *embedding table* is utilized to map a discrete, categorical feature into its corresponding vector representation, which is stored contiguously within the memory address space as a single dimensional array. Recommendation systems are formulated as a problem of predicting the likelihood of a certain event (e.g., the probability of a YouTube user watching

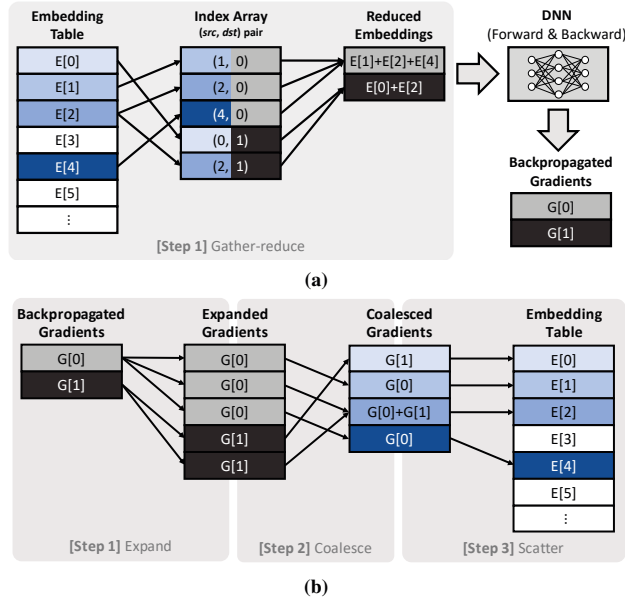


Fig. 2: High-level overview of (a) forward and (b) backpropagation of embedding layers. Example assumes a batch size of 2, each batch input gathering 3 and 2 embedding vectors, respectively. Although the embedding gather-reduce example in (a) appears as if it is a two separate steps (i.e., gather first using *src*, then reduce at *dst*), they are actually implemented as a *fused* kernel to save memory bandwidth. Concretely, the gather-reduce operator utilizes the *in-place* operative nature of reductions to conduct the gathered embedding vectors' reduction "on-the-fly" inside the on-chip registers.

a recommended video clip). Figure 1 provides a high-level overview of a DNN-based recommendation system, which consists of a frontend embedding layer and a backend DNN layer. As there exists multiple categorical features (e.g., user ID, item ID, ...) that are helpful in capturing various semantic representations, multiple embedding tables are utilized.

B. Training DNN-based Recommendations

Forward propagation. Figure 2(a) illustrates the forward propagation step of training embedding layers in DNN-based recommendation models [41]. An embedding table contains millions to billions of embedding vectors, each of which is typically sized at several hundreds of bytes, rendering a single embedding table to amount to several tens of GBs. As multiple embedding tables are utilized for a recommendation model, the aggregate size of embedding tables can reach several tens to thousands of GBs [35], [41], [57]. The embedding *gather* operation utilizes an array of index IDs (provided as part of the training dataset) to lookup multiple rows from the tens of GBs of embedding table. While the embedding table itself is stored in memory in a dense manner, gathering embedding vectors exhibits a highly *sparse* and *irregular* memory access pattern with low locality. The gathered embedding vectors are then combined with each other through an element-wise operation (e.g., addition, multiplication, etc.), which is equivalent to a *reduction*. As illustrated in the example in Figure 2(a), the index array for gather-reduce consists of a (*src*, *dst*)

Algorithm 1 Gradient Coalescing

```

1: procedure COALESCING(src, exp_grad)
2:   ▷ Step-A: Sort src array to determine coalescable indices
3:    $n \leftarrow \text{length}(\text{src})$ 
4:    $\text{sorted\_pos} \leftarrow \text{ARGSORT}(\text{src})$ 
5:    $\text{sorted\_src} \leftarrow \text{SORT}(\text{src})$ 
6:   ▷ Step-B: Accumulate coalescable gradients
7:    $(i, \text{prev}) \leftarrow (-1, -1)$ 
8:   for  $j \leftarrow 0$  to  $n$  do
9:      $\text{pos} \leftarrow \text{sorted\_pos}[j]$ 
10:     $\text{curr} \leftarrow \text{sorted\_src}[j]$ 
11:    if  $\text{curr} \neq \text{prev}$  then
12:       $i \leftarrow i + 1$ 
13:       $\text{coal\_grad}[i] \leftarrow \text{expanded\_grad}[\text{pos}]$ 
14:    else
15:       $\text{coal\_grad}[i] \leftarrow \text{coal\_grad}[i] + \text{exp\_grad}[\text{pos}]$ 
16:    end if
17:  end for
18:  return  $\text{coal\_grad}$ 
19: end procedure

```

pair which is utilized to determine which element to lookup (using *src* ID) from the table and where to store the reduced embedding vector (using *dst* ID). The reduced tensor is then fed into backend DNN layers which is typically constructed using MLPs, followed by a softmax layer that derives the click-through rate (CTR), e.g., the likelihood of an end-user clicking an Ad banner. The backpropagation step of DNN layers then generates a set of gradient vectors to backpropagate into embedding layers. The number of gradients to backpropagate is equivalent to the number of reduced embeddings during forward propagation, which is 2 (i.e., $G[0]$ and $G[1]$) in the given example.

Backpropagation. Training involves updating the model parameters of the DNN, which is done using stochastic gradient descent (SGD) [38]. Unlike conventional dense DNNs, an interesting property of ML applications utilizing embedding layers is that the contents inside the embedding tables (i.e., the embedding vectors) are crucial part of the model that are being trained. Consequently, the embeddings that have been gathered and subsequently reduced during forward propagation (e.g., *src* index ID 1, 2, and 4 for the first batch and ID 0 and 2 for the second batch in Figure 2) must be updated (i.e., trained) during backpropagation. Figure 2(b) shows how the two gradients, backpropagated from the backend DNN layers, are used to update multiple rows within the embedding table (i.e., $E[0, 1, 2, 4]$, those that have been looked up during forward propagation), which is effectively a gradient *scatter* operation back to those *src* indices used for embedding gathers (i.e., the gradient scatter operator is the *dual* operation of tensor gather for backpropagation). Note that, a given embedding vector can be read out multiple times for reduction operations across *different* batches (e.g., embeddings for *src* ID=2 is gathered twice for the first and second batch). Under such circumstances, the corresponding embedding vector has *multiple* gradient values to account for during model updates. That is, the multiple gradients must be accumulated (or *coalesced*) first before being utilized for updating the model parameter, i.e., $G[0] + G[1]$ is used for

updating the embedding vector $E[2]$. A key reason why gradients are not instead *sequentially* updating the model directly (which obviates the need for gradient coalescing) is because ML frameworks are designed to support a variety of optimization algorithms (e.g., RMSprop, Adagrad, momentum, ...), which require the (potentially multiple) gradients for updating a given model parameter at the i^{th} iteration (W_i) to first be accumulated into a single value (G_i), as exemplified through the optimization functions of RMSprop (Equation 1) and Adagrad (Equation 2) model updates.

$$A_i = \gamma A_{i-1} + (1-\gamma)G_i^2, \quad W_i = W_{i-1} - lr \times \frac{G_i}{\sqrt{\epsilon + A_i}} \quad (1)$$

$$A_i = A_{i-1} + G_i^2, \quad W_i = W_{i-1} - lr \times \frac{G_i}{\sqrt{\epsilon + A_i}} \quad (2)$$

As illustrated in Figure 2(b), the gradient vector for batch 0 and batch 1 are first each *expanded* to 3 and 2 vectors respectively (i.e., the gradient expand operator is the *dual* operation of tensor reduce for backpropagation), equivalent to the number of gathers conducted during forward propagation. The expanded vectors (i.e., tensors) are then *coalesced* into a single vector by examining whether the two tensors have any overlapping sparse indices (e.g., $src\ ID=2$ is used for both batch 0 and 1) used for constructing the gather-and-reduced vectors during forward propagation. Algorithm 1 details the two-step procedure of how the expanded gradients are coalesced. First, the *src* part of the index array of Figure 2(a) is *sorted* (line 2–5)² to have the coalescable indices appear as consecutive elements after sorting (i.e., $[1, 2, 4, 0, 2] \rightarrow [0, 1, 2, 2, 4]$, line 5). Then, the sorted indices are utilized for *accumulating* the gradient values sharing a common *src* ID, deriving the coalesced gradients (line 6–17). In this paper, we collectively refer to the aforementioned procedure as *gradient expand-coalesce* operation³, which generates the coalesced gradients (G_i) to be used by the optimization function for the *scatter* operation to update the models inside the embedding table.

C. System Architectures for Training

This paper assumes GPUs as the baseline TPU architecture as it is currently the most popular platform for training purposes. Nonetheless, our proposal is equally applicable for other GEMM-optimized TPUs. State-of-the-art TPUs for training [15], [16], [21], [28], [43] typically utilize bandwidth-optimized 3D stacked DRAMs such as HBM [29]. Stacked DRAMs however are capacity limited, only available with several tens of GBs. Consequently, existing solutions for training recommendations almost exclusively utilize either the host CPU memory [14], [19], [57] or a disaggregated memory pool [10], [35] populated with capacity-optimized DRAM (e.g., DDR4) as a separate placeholder to store the memory-hungry embedding tables. Two possible system

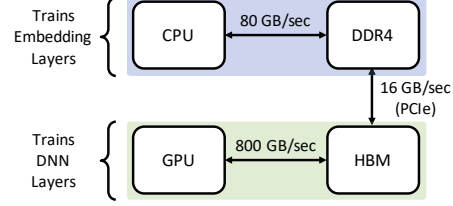


Fig. 3: System architecture for training recommendation models.

solutions for training recommendations are as follows. A *CPU-only* solution (CPU-only) conducts training using the CPU without GPUs [20]. A *CPU-GPU* based approach (CPU-GPU) utilizes the CPU for executing the embedding layers while the DNN is handled by the GPU [18], [35], [57]. A common property of both solutions is that the key primitives of training embedding layers (i.e., tensor gather-reduce, gradient expand-coalesce, and gradient scatter, detailed in Figure 2) are all conducted over the CPU. We henceforth collectively refer to both CPU-only and CPU-GPU as *CPU-centric* training systems as the embedding layers are trained using the CPU (Figure 3).

D. Related Work

There have been several prior literature within the computer system community focusing on the computationally intensive CNNs, RNNs, and MLPs, which exhibit a dense and highly regular computational property. Because of its highly deterministic dataflow, these dense DNN algorithms are amenable for accelerated computation using custom-designed architectures for both training and inference [6], [7], [8], [9], [22], [26], [34], [36], [37], [49], [51], [52]. Unlike these prior art, DNN-based recommendation models are a new type of ML workload that has only recently been the target of study in the academic literature. Recent work provides a comprehensive characterization study on CPU-only [19], [20], [24] and CPU-GPU [35], [57] based inference system for personalized recommendations. As discussed in Section II-C, both studies assume the host CPU memory stores the memory-hungry embedding tables as assumed in our baseline system architecture. More relevant to our study is work by Kwon et al. [35] and Ke et al. [32] which proposes a near-memory processing architecture for recommendation inference. As we detail in Section III-A, these prior work is specifically designed as a point solution to the inference pass of recommendations, so it is not able to effectively handle the gradient expand-coalesce primitives of embedding layer's backpropagation step. To the best of our knowledge, this work is the first to provide a detailed characterization study on training recommendations, revealing important system-level bottlenecks such as gradient expand-coalesce or gradient scatter. More crucially, our treatment of the gradient expand-coalesce primitive into a generic tensor gather-reduce operator using our Tensor Casting algorithm is a significant first step in developing a training system that comprehensively accelerates the forward and backpropagation of embedding layers using a single accelerator architecture.

²The `ArgSort` in line 4 is functionally identical to Numpy's `argsort()`, which returns the indices that would sort the input array.

³Both TensorFlow [56] and PyTorch [50] employ such gradient expand-coalesce based approach to generate the accumulated gradient values.

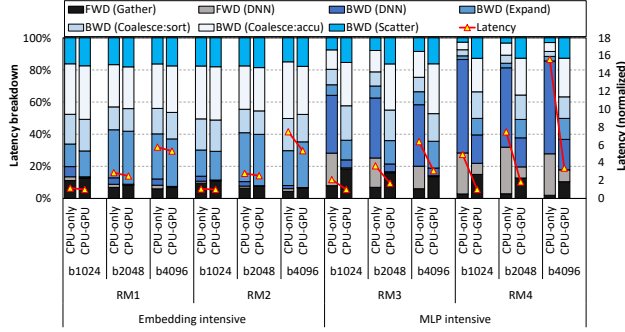


Fig. 4: Training time broken down into key steps of forward (black-gray) and backward propagation (blue) as a function of batch size (left-axis). Note that the latency of gradient coalescing (Algorithm 1) is broken down into the sorting (BWD (Coalesce:sort)) and accumulation step (BWD (Coalesce:accu)) for a detailed analysis. The training time normalized to the fastest configuration of each model (i.e., CPU-GPU with batch 1024), is shown on the right-axis.

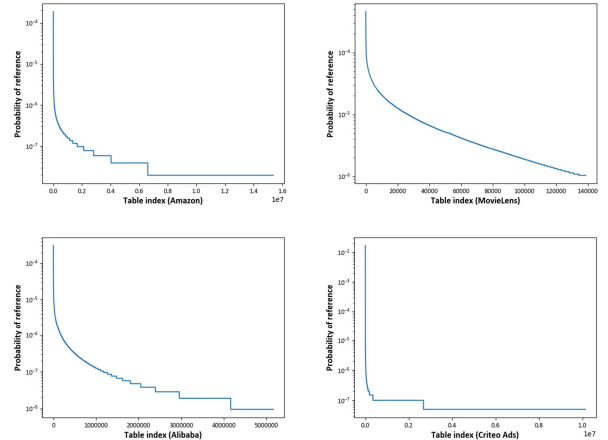
III. WORKLOAD CHARACTERIZATION

In this paper, we utilize the open-source deep learning recommendation model (DLRM) [41] as a vehicle to conduct a workload characterization study on training recommendations over CPU-only and CPU-GPU systems⁴. Similar to [19], [20], [32], we explore performance acceleration strategies for four model configurations (RM1/RM2 as embedding intensive configurations whereas RM3 and RM4 are limited by MLPs) representative of two canonical classes of recommendation models used for content filtering and ranking. The chosen models attribute to significant ML execution cycles at several hyperscaler’s datacenters [19], [32] and we use them to root-cause the performance limiters of training recommendations. Unlike CNNs or RNNs which are trained with a batch size of several hundreds, prior work on training DNN-based recommendation models typically employ several tens of thousands of mini-batches [13], [40]. Therefore, this paper assumes a default batch size of 2048 (the nominal batch size of DLRM) but sweeps this number from 1024 to 4096 to examine sensitivity. Section V further details our methodology.

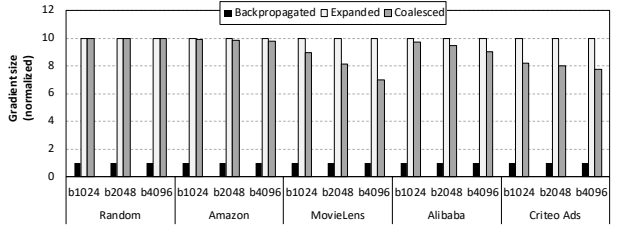
A. Breakdown of End-to-End Training Time

Figure 4 shows a breakdown of end-to-end training time (left-axis) as well as normalized latency (right-axis). Some key observations that can be made from this figure are as follows. First, there exists a noticeable performance gap between CPU-only and CPU-GPU, especially for MLP intensive RM3 and RM4, highlighting the significant role GPUs play in training recommendations. Interestingly however, MLPs account for only a small fraction of overall training cycles under CPU-GPU (less than 1% for embedding limited RM1/RM2 and 24% for MLP limited RM3/RM4), rendering the remaining embedding layers the most prominent performance bottleneck. Second, the backpropagation step

⁴Although *training* recommendations commonly employ CPU-GPU systems (unlike those for *inference* where both CPU-only and CPU-GPU are popular design points), we nonetheless study both designs in this section for completeness of our characterization.



(a)



(b)

Fig. 5: (a) Probability function that quantifies an embedding table entry’s likelihood of lookup per each dataset. (b) The size of gradient tensors before/after it is expanded and coalesced. Our experiment assumes each table is gathered 10 times, which is why the expanded gradient size is precisely $10\times$ larger than the initial backpropagated gradients. Results are normalized to the size of the backpropagated gradient tensor. *Random* assumes a uniform random distribution in modeling the probability of embedding table lookups.

of embedding layers accounts for approximately 62–92% of end-to-end training time. This shows the importance of understanding and properly accelerating the backpropagation step for training recommendations. Third, the tensor gather-reduce (forward) and gradient expand-coalesce primitive followed by gradient scatter (backward) account for the majority of training time, amounting to an average 60% and 93% for CPU-only and CPU-GPU, respectively. In particular, the gradient expand-coalesce takes up a substantial fraction of backpropagation’s latency, causing a significant bottleneck.

B. The Effect of Gradient Expand-Coalesce

As our characterization uncovered gradient expand-coalesce as causing a significant performance bottleneck, this subsection provides more detailed analysis on the important properties of this key primitive in relations to various recommendation training datasets and the data locality therein. As discussed in Figure 2, the gradient vectors backpropagated from the DNN layers must first be expanded and subsequently coalesced before the coalesced gradients are scattered back to the embedding tables. Note that the number of expanded gradients that should be coalesced is a function of how frequently the *same* sparse index ID (i.e., *src* ID=2 in Figure 2(a)) was used for embedding gathers during forward propagation. That is, the

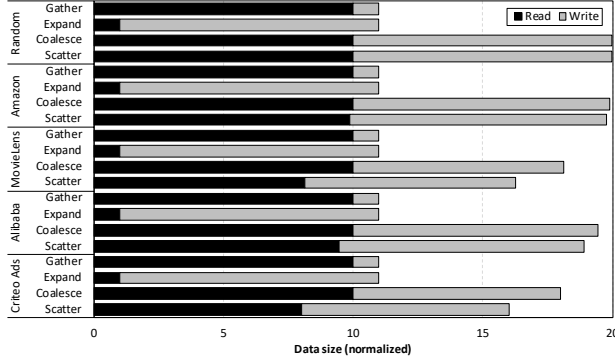


Fig. 6: Memory read/write traffic for key primitives in embedding layers. Similar to Figure 5, the analysis assumes each table is gathered 10 times.

more locality observed during the embedding gather process, the higher the likelihood of coalescing to reduce the size of the post-coalesced gradient tensor. To accurately reflect the locality inherent in embedding gathers for our characterization, we used publicly available training datasets for recommendations which include Amazon Review (Books) [4], MovieLens-20M [17], Alibaba’s TaoBao UserBehavior dataset [3], and Criteo AI Labs Ad Kaggle [31] to generate the sparse index IDs utilized for embedding table lookups. Using these publicly available datasets, we establish a histogram that counts the number of lookups for each distinct index IDs within a given embedding table. The sorted histogram is then utilized to generate the probability function of each embedding table entry’s likelihood of potential lookups. The recommendation datasets we study contain (several tens of) multiple tables, so for brevity, Figure 5(a) illustrates the probability function of the largest embedding table within each dataset. As depicted, a subset of table entries exhibit high access frequencies, which highlights the importance of the gradient coalescing step to accurately derive the accumulated gradient values for training. Utilizing the probability function in Figure 5(a), we measure the size of the backpropagated gradient vectors before and after they are expanded, and subsequently coalesced, the result of which is summarized in Figure 5(b). As we increase the training batch size, the likelihood of sparse indices for gathers “hitting” with each other is gradually increased as more table lookups are initiated. Consequently, the effectiveness of expanded gradient’s getting shrunk through coalescing is gradually increased as batch size gets larger. Even after coalescing however, there exists a non-trivial amount of coalesced gradient vectors that are subject for scatters, underscoring the importance of accurately modeling this crucial step for training.

C. Memory Intensity

To better understand the high latency overheads of backpropagation, this subsection analyzes the memory throughput demands of embedding layers. Section II-B discussed the memory intensive nature of key primitives in training embedding layers. To quantify the *microarchitecture independent* behavior

of embedding layer’s key primitives, we derive the amount of data the processor loads and stores for each primitive, which can be derived analytically by its algorithmic property. Figure 6 summarizes the memory read/write data traffic demands of the key primitives of embedding layers. Because the memory traffic demands of gradient coalesce is dominated by the gradient accumulation step (and not by the index array sorting step, see Algorithm 1), the “Coalesce” bar in Figure 6 only accounts for the gradient accumulation step within the gradient coalesce operator (i.e., only considers `BWD (Coalesce:accu)` and not `BWD (Coalesce:sort)` in Figure 4). In general, gradient coalesce and gradient scatter incur significantly higher memory traffic than gather-reduce (and correspondingly gradient expand), which explains the large fraction of training time spent conducting backpropagations. In particular, the gradient expand-coalesce step in aggregate incurs an around $3\times$ higher memory traffic than embedding gather-reduce. Furthermore, the sorting step in gradient coalescing (Algorithm 1) adds additional computation overhead on top of the already memory throughput limited gradient accumulation procedure. This explains why gradient expand-coalesce collectively accounts for a much more significant training time proportion when compared against other memory intensive primitives. Overall, our workload characterization on training recommendation models root-caused the backpropagation step of embedding layers, specifically the gradient expand-coalesce primitive, as a crucial system-level bottleneck.

IV. TENSOR CASTING: CO-DESIGNING ALGORITHM AND ARCHITECTURE FOR RECOMMENDATION TRAINING

We propose a vertically integrated solution encompassing multiple levels in the computing stack, ranging over algorithm (Section IV-A), software runtime system (Section IV-B) and architecture (Section IV-C). At the heart of our proposal is our novel *Tensor Casting* algorithm which “casts” the gradient expand-coalesce primitive into a tensor gather-reduce operator. The reason why our casting algorithm specifically targets tensor gather-reduce is twofold. First, Tensor casted (henceforth referred to as `T.Casted`) gather-reduce operation algorithmically reduces its memory intensity by $2\times$ compared to gradient expand-coalesce, improving performance. Second, it enables the development of a *generic* accelerator specifically optimized for tensor gather-reduce that encompasses *all* the key primitives of embedding layer training. In the rest of this paper, we assume CPU-GPU as the baseline system as the characterization in Section III demonstrated CPU-GPU’s superior performance compared to CPU-only.

A. Algorithm

Key observations. An important observation behind our Tensor Casting algorithm is that *coalescing* gradients is functionally equivalent to conducting *reductions* among the target gradients. If we were to think of the backpropagated gradient vectors as a “table” storing the gradients subject for coalescing, a gradient expand-coalesce operation is nothing more than *gathering* the necessary gradient vectors from

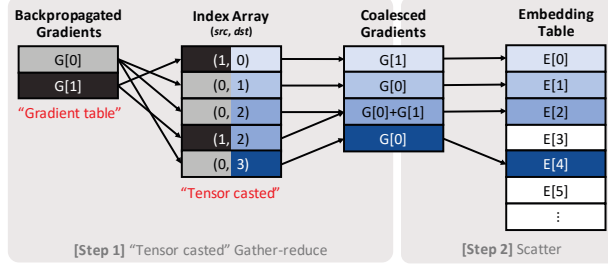


Fig. 7: The baseline gradient expand-coalesce “Tensor Casted” into our proposed gradient gather-reduce operation.

the “gradient table”, all of which are subsequently *reduced* (i.e., coalesced). Figure 7 provides a high-level overview of the effect of Tensor Casting, which transforms the two-step procedure of gradient expand and coalesce (Figure 2(b)) into a single, *fused* kernel call of `T.Casted` gather-reduce. Notice that the `T.Casted` gradient gather-reduce operation requires its own `T.Casted (src, dst)` pairs of index array to determine: 1) which gradient vectors to gather from the “gradient table” (using *src*), and 2) subsequently store the reduced gradients (using *dst*). This `T.Casted` index array is generated during the *casting* stage of our proposed algorithm (Section IV-B details when/where the casting step is conducted), which is detailed in Algorithm 2. We use Figure 8 as a driving example to highlight the important steps of Algorithm 2.

Implementation. As discussed in Section II-B, the gradient vectors subject for coalescing are for those embeddings that have been gathered-reduced across different batches during forward propagation (e.g., $E[2]$ in Figure 2). Consequently, the first step in Algorithm 2 *sorts-by-key* the original (*src, dst*) pairs of index array using the *src* ID as the *key* (line 3 in Algorithm 2). The goal of such sort-by-key procedure is to have those *src* indices gathered more than once during forward propagation to appear as consecutive elements within the sorted index array, allowing us to determine the *coalescable* indices (i.e., $ID=2$ in the *src* part of the sorted array, $[0, 1, 2, 2, 4]$). As the *dst* ID part of the sorted array pair now designates the batch ID number where the gather-reduced embedding vector was stored (see Figure 2(a)), we can utilize it as the *src* ID part of the `T.Casted` index array (line 4). In other words, the `T.Casted` gradient gather-reduce can utilize the *dst* ID part of the sorted array as its own `T.Casted src` ID to gather the necessary gradient vectors within the “gradient table” (i.e., the black-gray colored $[1, 0, 0, 1, 0]$ array in Figure 7). As the *src* part of the sorted index array now stores coalescable indices in consecutive index locations, we *scan* the sorted index array followed by a cumulative sum operation (line 5 – 9) to derive where the gathered-reduced gradients should eventually be stored within the coalesced gradient vector array. As a result, the final output array in Figure 8 can be utilized as the *dst* ID part of the `T.Casted` index array to determine where the reduced gradients should be stored (i.e., the $[0, 1, 2, 2, 3]$).

Algorithm 2 Tensor Casting

```

1: procedure TENSORCASTING(src, dst)
2:    $n \leftarrow \text{length}(\text{src})$ 
3:   (sorted_src, sorted_dst)  $\leftarrow \text{SORTBYKEY}(\text{src}, \text{dst})$ 
4:   casted_src  $\leftarrow \text{sorted_dst}$ 
5:   for  $i \leftarrow 1$  to  $n$  do
6:      $\text{scan}[i] \leftarrow (\text{sorted\_src}[i] \neq \text{sorted\_src}[i-1]) ? 1 : 0$ 
7:   end for
8:    $\text{scan}[0] \leftarrow 1$ 
9:   casted_dst  $\leftarrow \text{CUMULATIVESUM}(\text{scan}) - 1$ 
10:  return (casted_src, casted_dst)
11: end procedure

```

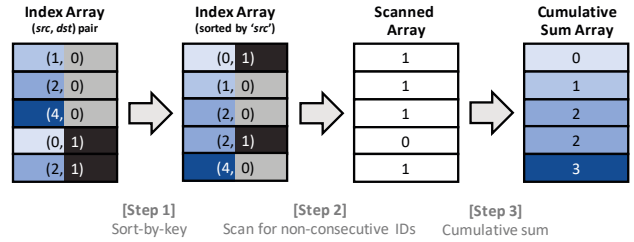


Fig. 8: Illustration of how our Tensor Casting algorithm derives the “`T.Casted`” (*src, dst*) ID array, which is required by the casted gradient gather-reduce operation in Figure 7.

Merits. A key advantage of `T.Casted` gradient gather-reduce is as follows. First, the two-step procedure of gradient expand and coalesce is now *fused* into a single gradient gather-reduce, significantly alleviating the high memory demands of this crucial bottleneck. Second, our proposal utilizes the hybrid nature of CPU–GPU system to hide the latency incurred during the casting stage of Tensor Casting. Note that the baseline gradient expand-coalesce conducts the sorting step (i.e., the process in which the coalescable gradients are determined, Algorithm 1) as part of the gradient coalescing operation. Such implementation renders the latency to determine the coalescable indices to be directly exposed as part of the backpropagation latency. Our unique observation is that the casting stage of Tensor Casting (Algorithm 2) can be completely *decoupled* from the `T.Casted` gradient gather-reduce operation, allowing us to intelligently hide the casting latency during forward propagation step. Below, we detail our proposed runtime system that is co-designed with the underlying system architecture to maximize the benefits of Tensor Casting.

B. Software Runtime System

Design principle. As discussed in Section II-C, the conflicting resource requirements of sparse embedding layers and dense DNN layers mandate a *hybrid* system architecture for training recommendations: a CPU–GPU system where the sparse embedding layers are trained on the CPU while the dense DNN layers are trained on the GPU (Figure 3). A rather unfortunate side-effect of such design point is that either one of the processor architecture remains idle when the other compute engine is busy executing (Figure 9(a)). Our software runtime system utilizes such unique property of hybrid CPU–GPU to our benefit by conducting the Tensor Casting’s casting step (i.e.,

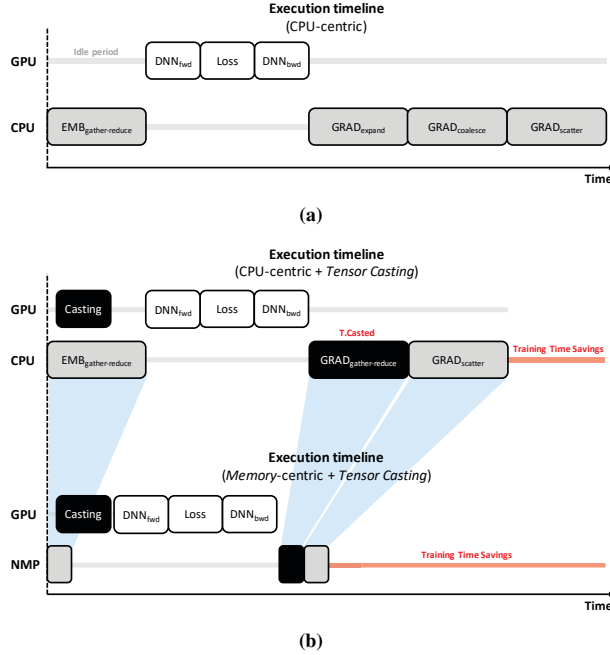


Fig. 9: Execution timeline of (a) CPU-centric baseline and (b) our proposed CPU-centric and memory-centric system using Tensor Casting.

Algorithm 2) during the forward propagation stage as depicted in Figure 9(b). An important observation from Algorithm 2 is that all the data structures required to generate the $T.Casted (src, dst)$ pair of index array is already available at the very beginning of forward propagation. As the GPU remains idle during the course of CPU-side embedding gather-reduce, our runtime system copies the original (src, dst) index array to the GPU over PCIe and utilize the GPU to proactively initiate the casting step of Tensor Casting. This allows our runtime system to immediately utilize the *precomputed* $T.Casted$ index array during backpropagation and conduct the $T.Casted$ gradient gather-reduce operation for training time savings. Algorithm 3 summarizes the aforementioned process, which utilizes the Tensor Casting algorithm (Algorithm 2) to convert the baseline, two-step process of gradient expand-coalescing (Figure 2) into a single-step gradient gather-reduce.

Overhead. As we detail in Section VI, the overhead of copying the index array is negligible as its size is only in the order of several MBs, even with several thousands of input batches. More importantly, the advantage of such GPU-side casting operation is that its latency can be hidden inside the CPU-side embedding gather-reduce operation. Overall, the proposed runtime system and Tensor Casting utilize the currently available software stack as-is, so an important benefit of our proposal is that it can be implemented purely in software over existing CPU-centric CPU-GPU systems. As we quantitatively demonstrate in Section VI-B, Tensor Casting applied on top of the baseline CPU-GPU provides 1.2–2.8× improvement in training throughput.

Algorithm 3 Tensor Casted Gradient Gather-Reduce

```

1: procedure GATHERREDUCE( $src, dst, grad$ )
2:    $n \leftarrow \text{length}(src)$ 
3:   for  $i \leftarrow 0$  to  $n$  do
4:      $coal\_grad[dst[i]] \leftarrow coal\_grad[dst[i]] + grad[src[i]]$ 
5:   end for
6:   return  $coal\_grad$ 
7: end procedure
8:
9: procedure T.CASTEDGRADGATHERREDUCE( $src, dst, grad$ )
10:  ▷ Step A: Execute Tensor Casting algorithm (Algorithm 2)
11:  ( $casted\_src, casted\_dst$ )  $\leftarrow$  TENSORCASTING( $src, dst$ )
12:  ▷ Step B: Initiate GatherReduce kernel for Gradient Updates
13:  return GatherReduce( $casted\_src, casted\_dst, grad$ )
14: end procedure

```

C. Architecture

We have so far discussed the merits of co-designing the training algorithm under the CPU-centric CPU-GPU systems. Nonetheless, conventional CPU-GPU systems do not fully reap out the full potential existent with our Tensor Casting algorithm, as the memory-limited primitives of embedding layers are still conducted over the memory-bandwidth limited CPU memory subsystem. To quantify the maximum performance improvements possible with our Tensor Casting, this subsection presents our *memory-centric* approach for training recommendations. Before we detail our proposed architecture, let us first discuss some important properties and requirements of accelerator architectures for training.

Architectures for training “dense” DNNs. Because SGD based training involves the derivation of input and weight gradient vectors, a robust training architecture should be able to cover the important primitives of both forward and backward propagation. Consequently, TPUs for training are typically optimized for GEMM operators because of the versatility of this all-round compute primitive [15], [16], [21], [43], [55]. Concretely, the compute primitives of both forward and backward propagation of popular dense DNN layers (e.g., convolutional, fully-connected, and recurrent layers) are able to be “casted” as GEMMs, amortizing the design cost of TPUs by specializing its microarchitecture for GEMMs. As such, an important design principle in TPU architectures for training DNNs is maintaining its applicability to key primitives of both forward and backward propagation while also achieving the high energy-efficiency of specialized accelerators.

Architectures for training “sparse” embeddings. Our workload characterization has root-caused embedding gather-reduce (forward), gradient expand-coalesce, and gradient scatter (backward propagation) as the three most time-consuming primitives of training embedding layers. Given the increasing importance of this memory-limited ML algorithm and the significant performance bottleneck it incurs, we argue that ML workloads utilizing embeddings deserve a targeted acceleration treatment. Tensor Casting has been carefully designed from the ground up to enable the design of a *generic* accelerator architecture meeting the aforementioned two design requirements of training: its applicability to both forward and backpropagation and delivering high-energy efficiency through

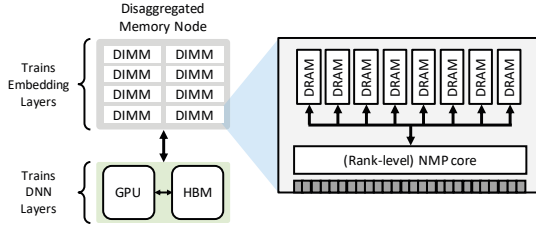


Fig. 10: Proposed memory-centric system for training recommendation models.

specialized microarchitecture designs. Recall that Tensor Casting can transform the gradient expand-coalesce primitives into gradient gather-reductions. Additionally, notice that the datapath of scatter operations is virtually identical to gathers as both operations can be conducted over the same datapath, just in the opposite directions (detailed later using Figure 11). As such, the development of our Tensor Casting algorithm to transform gradient expand-coalesce into gather-reduce is no coincidence, as we seek to *unify* all key primitives of training embeddings under a *single* compute operator. Consequently, our proposition is to optimize the accelerator microarchitecture using *near-memory processing* (NMP) technology for high-throughput tensor gather-reductions as well as tensor scatters. Building on top of recent DIMM-based NMP accelerator designs [1], [2], [5], [32], [35], the **key innovation** of our proposal is the utilization of the *expressive power of the tensor gather-reductions* to architect a sparse-optimized accelerator for training embedding layers. The merits of an NMP-based tensor gather-scatter accelerator are clear: 1) the design can cover the majority of embedding training time using a single microarchitecture, and 2) the NMP design paradigm can fundamentally address the memory throughput bottlenecks of embedding layers.

A “memory-centric” recommendation training. There have been several prior studies [1], [2], [5], [32], [35] exploring the advantages of augmenting commodity DIMM devices with NMP accelerators optimized for specific application domains. In particular, prior work by Kwon et al. [35] and Ke et al. [32] each explored the merits of NMP acceleration for CPU-GPU and CPU-only based systems for recommendation inference. As discussed in Section II-C, systems for training typically utilize GPU for accelerating dense DNN layers, so we discuss how Tensor Casting can be employed on top of the GPU-centric disaggregated memory system as suggested by Kwon et al. Nonetheless, Tensor Casting can be readily employed under the CPU-centric NMP accelerator proposed by Ke et al. because the key intuition behind our proposal (i.e., utilizing the expressiveness of tensor gather-reduce for accelerating both forward/backward propagation) is orthogonal to the underlying CPU vs. GPU based NMP microarchitecture.

Figure 10 provides a high-level overview of our memory-centric approach in training recommendations. Here, the acceleration of sparse embeddings and the dense DNNs is separated across the disaggregated, pooled memory architecture and the GPU, respectively. The disaggregated memory pool is

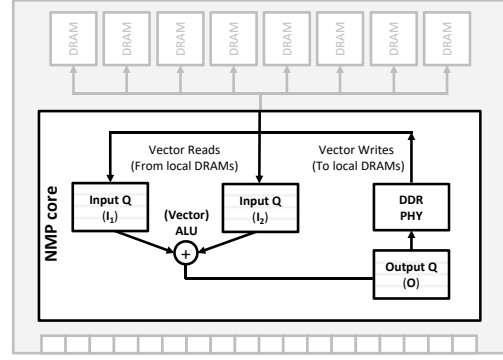


Fig. 11: NMP microarchitecture for tensor gather-scatter.

populated with multiple units of custom designed DIMMs, each of which is augmented with a NMP accelerator to be able to *locally* train the embedding tables. The aggregate memory capacity provided with the pool memory architecture can be several TBs when utilizing the latest capacity-optimized LR-DIMMs [14], [25], allowing the entire embedding tables to be stored locally. For training DNNs, the input tensor is copied over to the GPU to leverage its abundant compute and memory throughput. In terms of the NMP accelerator microarchitecture, our proposed design builds on top of the work by Kwon et al. [35] and Ke et al. [32] as detailed below.

NMP microarchitecture. Figure 11 details our NMP core design, each with 1) a vector ALU conducting the reduction among the gathered embeddings, 2) a local memory controller that translates the tensor gather-reduce and scatter instructions into low-level DRAM commands, and 3) a set of input/output buffers that stage in/out the embedding vectors or other metadata required for tensor gather-scatter. Similar to the design suggested by Kwon et al. [35], we assume that the GPU sends a CISC instruction encapsulating the necessary information required to conduct tensor gather-reduce (and similarly scatter), which the NMP core receives to conduct the necessary transactions locally within the DIMM. To support fine-grained embedding gather and scatter operations, the NMP core is equipped per each rank to utilize rank-level parallelism for bandwidth amplification. As the minimum access granularity per each rank is 64 bytes, each NMP core is able to conduct multiples of 64 byte granularity gathers and scatters. The multiple embedding tables are interleaved across the ranks such that the *effective* memory throughput available across the NMP cores are amplified as a function of the number of ranks employed within the disaggregated memory. The NMP local input/output buffers are utilized to conduct an on-the-fly reductions or temporarily store the data read out of DRAM, the result of which can be drained back to memory to store the reduced embeddings or for scatter operations. Because the NMP-based tensor gather-scatter is conducted without having to move the data outside the DIMMs, our solution enables significant increase in the aggregate effective bandwidth than conventional pin-limited memory subsystems. Further details regarding the design of an NMP unit for gather-reduce, the required system software support and API extensions are

TABLE I: Disaggregated memory architecture configuration.

DRAM specification	DDR4
Number of ranks	32
Effective memory bandwidth (per rank)	25.6 GB/sec
Effective Memory bandwidth (in aggregate)	819.2 GB/sec

available at [32], [35] and we refer the interested readers to these prior work due to space constraints.

D. Putting Everything Together

This section described the key facets of our algorithm and architecture co-design which targets recommendation training systems. Key innovation of our proposal is our novel Tensor Casting algorithm, which enables the acceleration of training embedding tables using a *single* compute primitive – tensor gather-reductions. Based on such insight, we propose, implement, and demonstrate our proposal on top of an existing CPU-centric system as well as a more future-looking memory-centric system architecture, achieving significant training throughput improvements (Figure 9).

V. METHODOLOGY

Evaluation framework. Our software runtime is designed using the open-sourced PyTorch backend library (v1.5.0) for modeling embedding layers and NVIDIA’s cuDNN/cuBLAS/Thrust [42], [46], [47] and Intel MKL [27] for DNN layers. To establish a strong baseline to compare against our proposal, we characterize the performance of the key primitives of PyTorch library and heavily optimize and tune its performance whenever necessary, for a conservative analysis. For instance, our tuned version of gradient coalesce achieves $5.0 - 6.1\times$ and $6.7 - 12\times$ higher performance than baseline PyTorch for the sorting and accumulation steps of gradient coalescing (Algorithm 1), respectively, by better parallelizing and tuning its execution⁵. All the evaluation conducted in this paper (including our motivational data in Figure 4) utilizes our optimized implementation of PyTorch. As Tensor Casting is purely an algorithmic innovation which can be implemented completely in software, we utilize CUDA as well as existing software libraries (e.g., NVIDIA’s NVlabs CUB [48]) and PyTorch runtime APIs to design our proposed algorithm. We thoroughly validate the functional equivalence between the baseline gradient expand-coalesce primitive and our proposed tensor casted gradient gather-reduce operator.

As our CPU-centric system with Tensor Casting can be evaluated over real systems, we measure the end-to-end wall clock time for reporting performance. To quantify the benefits provided with our NMP based memory-centric system, we follow the methodology suggested by Kwon et al. [35] which employs an emulation based study. As the key primitives of embedding layers (e.g., tensor gather-reduce) are memory bandwidth limited, these prior work utilize a cycle-level DRAM simulator to measure the effective memory throughput of the memory system when fed in with the appropriate DRAM

⁵TensorFlow’s implementation of the gradient expand-coalesce operator exhibited similar performance bottlenecks, so we utilize our optimized/tuned version of PyTorch as baseline for a conservative analysis.

TABLE II: Recommendation model configurations.

Model	# of Tables	Gathers/table	Bottom MLP	Top MLP
RM1	10	80	256-128-64	256-64-1
RM2	40	80	256-128-64	512-128-1
RM3	10	20	2560-512-64	512-128-1
RM4	10	20	2560-1024-64	2048-2048-1024-1

commands. The effective memory throughput is then utilized as a proxy to emulate the performance of the configured memory subsystem when executing the NMP operations over real systems as follows. Our emulated disaggregated memory architecture is assumed as employing enough number of ranks to deliver an aggregate effective memory bandwidth commensurate to that of a GPU’s local HBM memory bandwidth (Table I). We then use Ramulator [33] to model our emulated disaggregated memory architecture, which achieves significant memory throughput thanks to its near-memory processing nature (over 600 GB/sec of effective throughput over the maximum 819.2 GB/sec). The NMP-enhanced tensor gather-reduce and scatter functions (Figure 9(b)) are then implemented as a CUDA kernel that emulates the behavior of NMP operations over a real GPU (one which is assumed as the disaggregated memory pool), which we integrate into our software runtime system to measure end-to-end performance of training recommendations. In general, our emulation methodology follows that of [32], [35].

System configuration. We utilize NVIDIA’s DGX system [44] equipped with eight V100 GPUs [43] for our study. The NVIDIA V100 comes with 900 GB/sec of memory throughput (similar to our 32 ranked disaggregated memory, Table I) with six NVLINKs for communicating with the other GPUs up to 150 GB/sec. A single GPU card communicating with the host CPU over PCIe(gen3) is assumed when evaluating the CPU-GPU baseline and our Tensor Casting CPU-GPU. For our memory-centric system, we utilize a pair of V100s to model the NMP-GPU system, where one of the GPUs emulates the behavior of our NMP-augmented disaggregated memory node. We configure the communication bandwidth between NMP-GPU to be 25 GB/sec, which is closest we could configure to be commensurate to the PCIe(gen3) bus bandwidth utilized for CPU-GPU. As we detail in Section VI-D, the performance of NMP-GPU is insensitive to the communication bandwidth.

Benchmarks. We study four recommendation models using the open-sourced DLRM [41] (Table II). RM1-3 are configured identically as discussed by Gupta et al. [19], with RM1-2 exhibiting embedding intensive and RM3 showing MLP intensive behavior. RM4 is modeled as an MLP intensive workload by stacking one additional MLP layer and increasing all MLP layer’s parameters. Following prior work [40], [41], the default embedding vector size is set as a 64-dimensional vector. As recommendations are typically trained with a batch size of several thousands to tens of thousands of inputs, we study a batch size of 1024/2048/4096 as our default setting. In Section VI-D, we study the sensitivity of Tensor Casting when deviating from these default configurations.

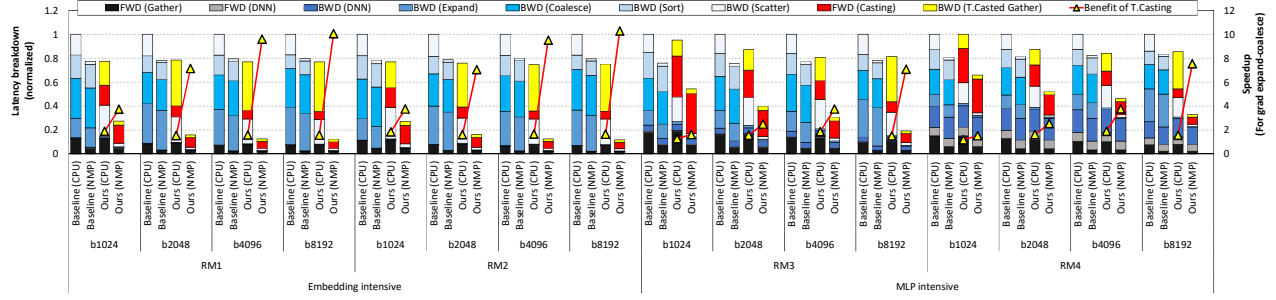


Fig. 12: Latency breakdown of our studied workloads. The left-axis shows a stacked bar chart of the execution time of each individual compute primitives (normalized to Baseline(CPU)). The right-axis shows the speedup Tensor Casting brings about just for the gradient expand-coalesce step by measuring (execution time of gradient expand-coalesce)/(execution time of the casting step (red) and T.Casted gather-reduce (yellow)). Baseline(NMP) models a TensorDIMM based NMP design which can accelerate “both” embedding gather-reduce and gradient scatters, but gradient expand-coalesce is executed without Tensor Casting, identically as done in Baseline(CPU).

VI. EVALUATION

We explore four system design points: the 1) baseline CPU-centric system utilizing CPU-GPU as-is (Baseline(CPU)), 2) TensorDIMM-based baseline NMP accelerator which can accelerate both embedding gather-reduce and gradient scatters, but gradient expand-coalesce being executed identically as baseline CPU-centric system without Tensor Casting (Baseline(NMP)), 3) our Tensor Casting applied CPU-centric system (Ours(CPU)), and 4) our proposed memory-centric system utilizing both NMP and Tensor Casting (Ours(NMP)). Note that Tensor Casting *does not change the algorithmic nature of SGD training* as our proposal does not change the mathematical property of gradient coalescing. Hence, the total number of training iterations required to reach the same level of CTR accuracy of a recommendation is *identical* between baseline and our proposal.

A. Latency Breakdown

As discussed in Section IV-B, our software runtime intelligently hides the casting overhead of Tensor Casting by initiating this process during forward propagation. Before we discuss the end-to-end performance speedup provided with Tensor Casting, this subsection first quantifies the efficacy of Tensor Casting in addressing the performance bottlenecks of the baseline gradient expand-coalesce primitive. Figure 12 shows a latency breakdown of our studied workloads, which is a stacked bar chart of the execution time of each individual compute primitives during training (left-axis). While the accumulated latency is indicative of performance, it does not exactly reflect the end-to-end training time as our runtime can hide the latency of the casting stage of Tensor Casting. As discussed in Section III-A, the baseline CPU-GPU consumes significant latency during the backpropagation stage, with the gradient expand-coalesce operator incurring the most noticeable latency overhead. Tensor Casting substantially reduces the time taken to conduct this time-consuming step by permuting it into gather-reduce, which is represented by the much smaller time taken to execute the casting (red) and the actual T.Casted gather-reduce (yellow), achieving

1.1–9.5 $\times$ latency reduction for this bottleneck operator (right-axis). As we detail in Section VI-B, the end-to-end training time reduction is even higher as our runtime system hides the latency of the casting stage during forward propagation. As for our memory-centric system, the time taken to conduct the T.Casted gradient gather-reduce operation is further reduced by an additional 1.3–6.1 $\times$ compared to Ours(CPU) (1.5–9.5 $\times$ speedup than Baseline(CPU)) all thanks to our versatile NMP cores. In fact, the performance advantage of NMP is so pronounced that the casting stage can sometimes become a new performance bottleneck under our memory-centric system (i.e., the latency overhead of casting is identical between Ours(CPU) and Ours(NMP)).

B. System-level Performance

Figure 13 summarizes the end-to-end speedup offered with our proposal. In general, as Tensor Casting primarily improves the performance bottlenecks of embedding layers, the achieved speedups are more pronounced on embedding intensive RM1-2 than the MLP intensive RM3-4. Our software-only Tensor Casting running on top of hybrid CPU-GPU provide 1.2–1.6 $\times$ speedup under our default configurations, with 1.4–2.8 $\times$ speedup when trained with larger mini-batch sizes (Section VI-D). It is worth pointing out that our software-only Tensor Casting (Ours(CPU)) performs even better than the baseline TensorDIMM-based NMP accelerator (Baseline(NMP)), achieving an average 15% speedup. While the baseline NMP can utilize near-memory acceleration to reduce latency in conducting embedding gather-reduce and gradient scatter, the bottleneck incurred by gradient expand-coalesce still remains as the biggest performance limiter. These results highlight the importance of properly addressing the critical system-level challenges of embedding layer’s backpropagation step, which our Tensor Casting algorithm successfully delivers.

Under our memory-centric system design, all the key primitives of embedding layer training (i.e., embedding gather-reduce, gradient expand-coalesce, and gradient scatter) achieve significant latency reduction as our co-designed algorithm (Tensor Casting) and architecture (NMP cores) successfully

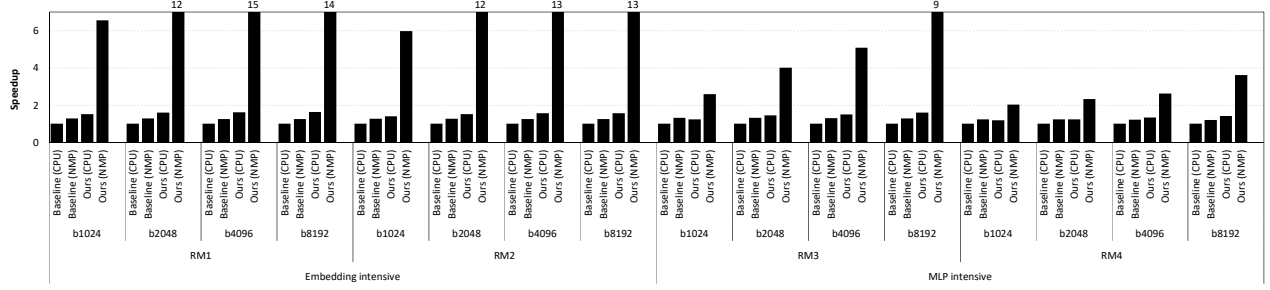


Fig. 13: End-to-end performance speedup provided with Tensor Casting.

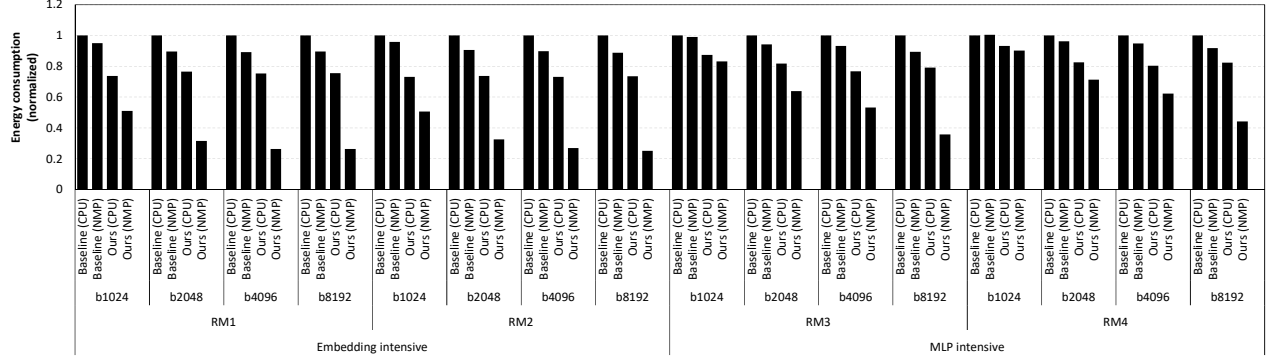


Fig. 14: Energy consumption with/without Tensor Casting.

accelerates these operations under our “unified” tensor gather-scatter unit. Overall, our memory-centric Tensor Casting approach achieves $2.0\text{--}15\times$ (average $6.9\times$) training throughput increase, significantly improving the state-of-the-art in training recommendation models.

C. Design Overheads, Energy-Efficiency, and NMP Utilization

Design overheads. Our software-only Tensor Casting algorithm is implemented purely on top of the existing hardware/software and can readily be deployed over real silicon. As our CPU-centric Ours (CPU) operates over real systems, the training time reduction directly translate into energy-efficiency (shown in Figure 14). Our NMP microarchitecture builds upon the DIMM-based NMP substrate as proposed in [32], [35]. Compared to these prior works, the modifications required in the DIMM or the underlying NMP microarchitecture is practically negligible as the key innovation of Tensor Casting is our permutation algorithm itself that enable *all* major compute primitives of training embeddings to operate over the tensor gather-scatter accelerator. The primary change required is the inclusion of the tensor scatter instruction as part of the ISA. We implement and synthesized the major components of our NMP core on top of a Xilinx Virtex FPGA board using Verilog HDL, confirming that the added area and power overheads of the NMP unit is practically negligible, which is in line with the conclusions from prior works [32], [35].

Energy-efficiency. We utilize powerstat to measure CPU’s socket-level power consumption and NVIDIA’s nvidia-smi for GPU power measurements. To quantify the power overhead of our NMP-augmented disaggregated

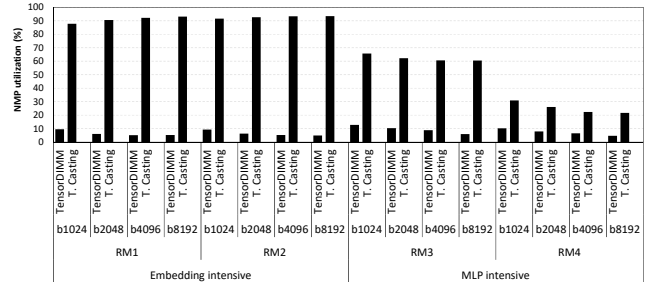


Fig. 15: NMP utilization (i.e., fraction of training time when NMP is active).

memory architecture, we follow the methodology employed in [35] utilizing Micron’s DDR4 system power calculator [39] to quantify the 32 ranked disaggregated memory node (assuming a 128 GB LR-DIMM DDR4 module per each rank), which we augmented with the aforementioned Verilog HDL based power estimation of NMP units. When evaluating energy consumption, we multiply the power estimation values with each CPU, GPU, and NMP node’s execution time. Figure 14 shows the energy-efficiency of Ours (NMP), where the significant training throughput improvements directly translate into energy savings. Notice that even the software-only Ours (CPU) provides noticeable energy-efficiency improvements compared to Baseline (NMP), underscoring the importance of properly addressing the bottlenecks of gradient expand-coalesce.

NMP utilization. To evaluate how well the NMP accelerator is utilized, with/without Tensor Casting, Figure 15 measures the fraction of training time when NMP is active. As depicted,

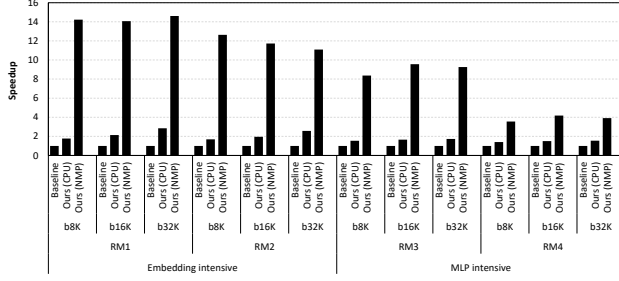


Fig. 16: Tensor Casting sensitivity to training batch size.

the baseline NMP *without* Tensor Casting (i.e., TensorDIMM) is only useful in accelerating embedding gather-reduce and gradient scatter operations (see Figure 12). As such, the NMP core is only active during an average 7% of training time (i.e., the NMP is left idle during 93% of the training time). Our Tensor Casting algorithm significantly improves the utilization of the NMP because it enables all major primitives of both forward and backpropagation for accelerated execution. Specifically, the NMP accelerator *with* Tensor Casting is actively utilized for an average 92% of training time for embedding intensive RM1/2 and an average 44% for MLP intensive RM3/4, a significant increase in NMP utility compared to TensorDIMM (which only achieves an average 6.5% and 8.5% utilization for RM1/2 and RM3/4, respectively), further justifying the design overheads of NMP architectures.

D. Sensitivity

Training batch size. While our nominal training batch size is chosen between 1024 to 4096, several prior works employ several tens of thousands of input batch sizes for training recommendations [13], [40]. Figure 16 summarizes the performance improvements Tensor Casting achieves with larger batch sizes, exhibiting up to $15\times$ throughput increase. In general, we observe that the effectiveness of Tensor Casting remains robust across a wide range of training batch sizes.

Embedding dimension size. While our baseline embedding vector dimension is configured as 64, there are prior works employing smaller [13] or larger [11] embedding vector dimensions than our assume embedding size of 64. Figure 17 summarizes the speedup provided with Tensor Casting under alternative embedding vector widths, achieving significant speedups and demonstrating the robustness of our proposal.

Communication bandwidth. For a conservative analysis, our default memory-centric system assumes a modest 25 GB/sec of communication bandwidth between the GPU and disaggregated memory. The goal of this sensitivity study is to sweep the available communication bandwidth (25 – 150 GB/sec) to explore how much performance is left on the table with a advanced communication protocols such as NVLINK [45]. We observe that even with our conservative baseline with PCIe(gen3) level communication, Tensor Casting already achieves 99% of the performance of a much more aggressive 150 GB/sec configurations, highlighting the robustness of Tensor Casting. We omit the results for brevity.

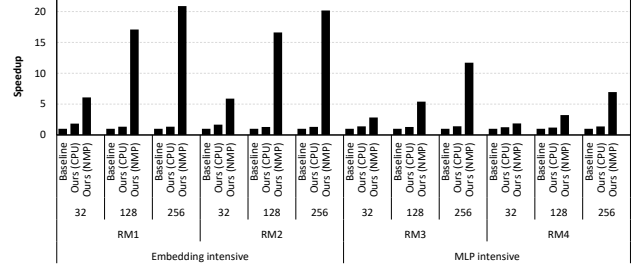


Fig. 17: Tensor Casting sensitivity to embedding vector size.

VII. CONCLUSION

This paper proposes Tensor Casting, an algorithm-architecture co-design for training recommendation models. Unlike recent prior literature focusing on the inference part of this emerging ML workload, the unique contribution of our study is the exploration of this application on the training side of things. We first provide a detailed, quantitative analysis on training recommendation models, root-causing several system-level bottlenecks such as gradient expand-coalesce. We then implement and demonstrate the benefits of Tensor Casting on real systems, showing that Tensor Casting achieves $1.9 - 21\times$ speedup than state-of-the-art approaches. To the best of our knowledge, Tensor Casting is the first that quantitatively explores architectural solutions tailored for training recommendation models.

ACKNOWLEDGMENT

This research is supported by Samsung Research Funding Center of Samsung Electronics (SRFC-TB1703-03), the Engineering Research Center Program through the National Research Foundation of Korea (NRF) funded by the Korean Government MSIT under Grant NRF-2018R1A5A1059921, and the funding supported by KAIST start-up package (G04180036). Minsoo Rhu is supported by the Facebook Faculty Research Award.

REFERENCES

- [1] M. Alian and N. S. Kim, “NetDIMM: Low-latency Near-memory Network Interface Architecture,” in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2019.
- [2] M. Alian, S. W. Min, H. Asgharimoghaddam, A. Dhar, D. K. Wang, T. Roewer, A. McPadden, O. O’Halloran, D. Chen, J. Xiong, D. Kim, W.-m. Hwu, and N. S. Kim, “Application-transparent Near-memory Processing Architecture with Memory Channel Network,” in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2018.
- [3] Alibaba, “User Behavior Data from Taobao for Recommendation,” <https://tianchi.aliyun.com/dataset/dataDetail?dataId=649>, 2018.
- [4] Amazon, “Amazon Product Data,” <http://jmcauley.ucsd.edu/data/amazon/>, 2018.
- [5] H. Asghari-Moghaddam, Y. H. Son, J. H. Ahn, and N. S. Kim, “Chameleon: Versatile and Practical Near-DRAM Acceleration Architecture for Large Memory Systems,” in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2016.
- [6] Y. Chen, J. Emer, and V. Sze, “Eyeriss: A Spatial Architecture for Energy-efficient Dataflow for Convolutional Neural Networks,” in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2016.
- [7] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, “DaDianNao: A Machine-learning Supercomputer,” in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2014.

- [8] Y. Choi, Y. Kim, and M. Rhu, "LazyBatching: An SLA-aware Batching System for Cloud Machine Learning Inference," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2021.
- [9] Y. Choi and M. Rhu, "PREMA: A Predictive Multi-task Scheduling Algorithm For Preemptible Neural Processing Units," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2020.
- [10] E. Choukse, M. Sullivan, M. O'Connor, M. Erez, J. Pool, D. Nellans, and S. W. Keckler, "Buddy Compression: Enabling Larger Memory for Deep Learning and HPC Workloads on GPUs," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2020.
- [11] P. Covington, J. Adams, and E. Sargin, "Deep Neural Networks for Youtube Recommendations," in *Proceedings of the ACM Conference on Recommender Systems (RECSYS)*, 2016.
- [12] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *arxiv.org*, 2018.
- [13] Facebook, <https://github.com/facebookresearch/dlrm>, 2019.
- [14] Facebook, "Accelerating Facebook's Infrastructure with Application-specific Hardware," 2019.
- [15] Google, "Cloud TPUs: ML Accelerators for TensorFlow," 2017.
- [16] Graphcore, "Intelligence Processing Unit," <https://www.graphcore.ai/dell-dss-8440-graphcore-ipu-white-paper>, 2019.
- [17] Grouplens, "Movielens 20M Dataset," <https://grouplens.org/datasets/movielens/>, 2016.
- [18] A. Gunny, C. Garg, L. Dolgovs, and A. Subramaniam, "Accelerating Wide & Deep Recommender Inference on GPUs," <https://devblogs.nvidia.com/accelerating-wide-deep-recommender-inference-on-gpus/>, 2020.
- [19] U. Gupta, S. Hsia, V. Saraph, X. Wang, B. Reagen, G.-Y. Wei, H.-H. S. Lee, D. Brooks, and C.-J. Wu, "DeepRecSys: A System for Optimizing End-to-end At-scale Neural Recommendation Inference," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2020.
- [20] U. Gupta, X. Wang, M. Naumov, C.-J. Wu, B. Reagen, D. Brooks, B. Cottel, K. Hazelwood, B. Jia, H.-H. S. Lee, A. Malevich, D. Mudigere, M. Smelyanskiy, L. Xiong, and X. Zhang, "The Architectural Implications of Facebook's DNN-based Personalized Recommendation," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2020.
- [21] Habana, "Habana Gaudi," <https://habana.ai/wp-content/uploads/2019/06/Habana-Gaudi-Training-Platform-whitepaper.pdf>, 2019.
- [22] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. Horowitz, and W. J. Dally, "EIE: Efficient Inference Engine on Compressed Deep Neural Network," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2016.
- [23] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural Collaborative Filtering," in *Proceedings of the International Conference on World Wide Web (WWW)*, 2017.
- [24] R. Hwang, T. Kim, Y. Kwon, and M. Rhu, "Centaur: A Chiplet-based, Hybrid Sparse-Dense Accelerator for Personalized Recommendations," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2020.
- [25] Hynix, "128 GB 3DS LRDIMM: The World's First Developed 3DS LRDIMM," 2017.
- [26] B. Hyun, Y. Kwon, Y. Choi, J. Kim, and M. Rhu, "NeuMMU: Architectural Support for Efficient Address Translations in Neural Processing Units," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operation Systems (ASPLOS)*, 2020.
- [27] Intel, "Intel Math Kernel Library," <https://software.intel.com/en-us/mkl>, 2019.
- [28] Intel, "Intel Nervana Neural Network Processor for Training," 2019.
- [29] JEDEC, "High Bandwidth Memory (HBM2) DRAM," 2018.
- [30] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Iuc Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmghani, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter Performance Analysis of a Tensor Processing Unit," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2017.
- [31] Kaggle, "Criteo Display Advertising Challenge," <https://www.kaggle.com/c/criteo-display-ad-challenge>, 2014.
- [32] L. Ke, U. Gupta, C.-J. Wu, B. Y. Cho, M. Hempstead, B. Reagen, X. Zhang, D. Brooks, V. Chandra, U. Diril *et al.*, "RecNMP: Accelerating Personalized Recommendation with Near-memory Processing," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2020.
- [33] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A Fast and Extensible DRAM Simulator," 2015.
- [34] Y. Kwon and M. Rhu, "A Disaggregated Memory System for Deep Learning," in *IEEE Micro*, 2019.
- [35] Y. Kwon, Y. Lee, and M. Rhu, "TensorDIMM: A Practical Near-memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2019.
- [36] Y. Kwon and M. Rhu, "A Case for Memory-Centric HPC System Architecture for Training Deep Neural Networks," in *IEEE Computer Architecture Letters*, 2018.
- [37] Y. Kwon and M. Rhu, "Beyond the Memory Wall: A Case for Memory-Centric HPC System for Deep Learning," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2018.
- [38] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [39] Micron, "Micron: System Power Calculator (DDR4)," 2017.
- [40] MLPerf, <https://github.com/mlperf/training>, 2019.
- [41] M. Naumov, D. Mudigere, H.-J. M. Shi, J. Huang, N. Sundaraman, J. Park, X. Wang, U. Gupta, C.-J. Wu, A. G. Azzolini, D. Dzhulgakov, A. Malleevich, I. Cherniavskii, Y. Lu, R. Krishnamoorthi, A. Yu, V. Kondratenko, S. Pereira, X. Chen, W. Chen, V. Rao, B. Jia, L. Xiong, and M. Smelyanskiy, "Deep Learning Recommendation Model for Personalization and Recommendation Systems," in *arxiv.org*, 2019.
- [42] NVIDIA, "Thrust Library," 2015.
- [43] NVIDIA, "NVIDIA Tesla V100," 2017.
- [44] NVIDIA, "The NVIDIA DGX-1V Deep Learning System," 2017.
- [45] NVIDIA, "NVLink High-speed Interconnect," 2018.
- [46] NVIDIA, "cuBLAS Library," 2019.
- [47] NVIDIA, "cuDNN: GPU Accelerated Deep Learning," 2019.
- [48] NVLabs, "CUB Library," 2018.
- [49] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2017.
- [50] PyTorch, <http://pytorch.org>, 2019.
- [51] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, and S. W. Keckler, "vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design," in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2016.
- [52] M. Rhu, M. O'Connor, N. Chatterjee, J. Pool, Y. Kwon, and S. W. Keckler, "Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks," in *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*, 2018.
- [53] A. Santoro, S. Bartunov, M. Botvinick, D. Wierstra, and T. Lillicrap, "Meta-learning with Memory-augmented Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2016.
- [54] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, V. Chandra, and H. Esmailzadeh, "Bit Fusion: Bit-level Dynamically Composable Architecture for Accelerating Deep Neural Networks," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2018.
- [55] C. System, "The Cerebras CS-1: the World's Most Powerful AI Compute System," <https://www.cerebras.net/>, 2019.
- [56] Tensorflow, <https://www.tensorflow.org>, 2016.
- [57] W. Zhao, J. Zhang, D. Xie, Y. Qian, R. Jia, and P. Li, "AIBox: CTR Prediction Model Training on a Single Node," in *Proceedings of the ACM International Conference on Information and Knowledge Management*, 2019.